

# Mini Internet Protocol Stack Simulator

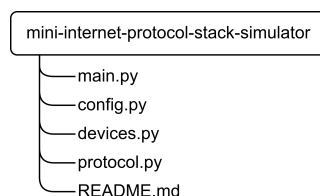
Lukas Rohwer (24215157), Harrison Chambers (24328083)

## Introduction

This document explains the structure and design of the code we wrote to simulate a mini Internet Protocol stack as described in the task sheet for the group project in CITS3002. We will begin by providing a general overview of the structure of the simulation, describing how each of the files work together, followed by an in-depth dive into the specific functions of each file and their purpose.

## Overview

As required by the task sheet, the project is split into four Python files accompanied by a README markdown file. The file structure is as follows:



Each file demonstrates a distinct purpose as briefly described below, and together form the simulation.

[main.py](#) is the main entry file, which is invoked from the command line along with an argument: an integer representing the size of the message in bytes. This file initiates the host and router objects representing devices in the network and begins the delivery of data from host A to host B.

[config.py](#) defines fixed parameters used in the simulation, including the IP addresses and MAC addresses of the devices, the routing tables used by the network layer, and ARP tables used by the link layer.

[devices.py](#) defines the Host and Router classes, including information about each, including their IP address, MAC address, router tables, their respective layer objects, as well as functions for receiving information from the application layer or the network. These are the classes defining the objects created in [main.py](#).

[protocol.py](#) contains the class definitions for the protocol headers Segment, Packet, and Frame, as well as the classes for Layers 2, 3, and 4 of the OSI model. Each layer class defines a function for sending and receiving data, passing it up or down the stack to the next layer.

[README.md](#) provides a cut-down explanation of the implementation of our project, along with instructions for running the program.

## In-Depth Explanation

Below is an in-depth explanation of the implementation of each file and how they fit together to simulate the transmission of data from one host to another.

### Config

In the config file, the fixed parameters used throughout the simulation are defined, which are imported into the rest of the files. First, the IP and MAC addresses of the hosts A and B are defined, which are used when creating the Host objects in [main.py](#). Next, the IP and MAC addresses of the two network interfaces are defined. These are used in [devices.py](#) when the router class defines its interfaces.

Next, the routing tables for host A, host B, and the router are defined. These map the destination IP address to the outgoing interface and the next-hop IP address. These tables are used by the network layer to determine how packets are forwarded across the network.

Finally, ARP tables are defined for each host and the router. These map the IP addresses of the next hop (which is provided by the network layer) to their respective MAC addresses. This table is used by the data link layer to determine the destination MAC address when sending a frame.

### Main

This is the main file used for testing and is the entry point into the simulation. It is invoked from the command line using the command `python main.py <message_size>`. 'message\_size' is a required argument defining the number of bytes in the message sent from the application layer.

The file imports the ``sys`` module to read the command line inputs, the Host and Router classes from [devices.py](#), and all parameters defined in [config.py](#). When the file is run, the ``main`` function is called, which first reads the command line inputs and returns an error message if there is not exactly one integer argument passed in. If the command line input is valid, a mock message from the application is created, consisting of a string of 'X's of size equivalent to the command line input.

Following this, each of the devices is initialised with the name of each device as well as parameters defined in [config.py](#). Next, the links between the devices are created so that a device can see the other devices connected to it. These links simulate wired or wireless links in a local network.

Finally, [main.py](#) invokes the 'application\_send()' function on host A which simulates host A sending a message, thus beginning the transfer of data from host A to host B. It passes in the data array, the origin and destination port, and the IP address of host B.

## Devices

The [devices.py](#) file defines the Host and Router classes, which are used in [main.py](#) to create the objects for each device in the network. Before it defines each class, it first imports the parameters defined in [config.py](#) and each of the header and layer classes defined in [protocol.py](#).

First, the host class is defined. It contains the name, IP and MAC address of the host, routing table, and ARP table, which were passed in from [main.py](#). It then initiates the sequence number and expected sequence number variables, which are used to add sequence numbers to outgoing segments and verify the order of incoming segments. An empty MAC table is created, which will record which MAC addresses a device has seen, simulating MAC learning. Objects for layers 2, 3 and 4 for each host are created using the class definitions in [protocol.py](#). Finally, the host class has two functions: one to simulate sending data from the application layer, and one to simulate receiving data from the link layer.

The router class similarly defines the name, routing table, ARP table, and MAC table. It then defines the two interfaces of the router along with their respective IP and MAC addresses from the config file. It then creates an instance of the network layer and link layer, and finally defines a function for receiving data from the link layer, from which data is passed up the stack.

## Protocol

[Protocol.py](#) defines the header data structures and layered protocol logic that simulate how data travels through the network stack, from the application layer down to the physical link layer and back up. It defines the three header structures - Segment, Packet, and Frame - required by the layers, as well as the layer architecture for the Transport Layer, Network Layer, and Link Layer.

### Header structures

The file defines a class for each of the three required headers:

#### **Segment header**

This represents a UDP-like Segment with ACK support. It contains the following attributes:

- Source and destination port numbers
- The length (size) of the segment (comprised by the size of the payload plus an 8-byte header).
- A checksum value
- The type of segment (either a '1' to represent an ACK segment, or a '0' to represent a Data segment)
- A sequence number for the segment
- The payload

### **Packet header**

This represents an IP-like Packet. It contains the following attributes:

- Source and destination IP addresses
- The current time-to-live (each hop lowers this by 1, stopping packets from hopping around until a random corruption issue)
- The protocol number
- Total length of the of the packet
- The segment as its payload

### **Frame header**

This represents an ethernet-like frame. It contains the following attributes:

- Source and destination MAC addresses
- The type of the frame (we use EtherType 0x800 to represent IPv4)
- The payload of the frame

Each of these classes are required to represent how data is encapsulated during the simplified network simulator.

### Layer architecture

The file defines a generic protocol layer parent class that stores a reference to the owning node. The subsequent three protocol layers (Transport Layer, Network Layer, and Link Layer) inherit from the Layer class. Each layer contains two functions, send and receive, that correspond with their respective functionality, as well as any further functions required by the layer. The following protocol layers are comprised of:

### **Transport Layer**

The Transport layer is comprised of its send and receive functions, as well as the following four functions:

- `segment_data` splits the data into chunks of at most 500 bytes (per the specification of the project) to fit within a single segment payload.
- `calculate_checksum` calculates the UDP checksum value for the payload (using the algorithm given to us in Week 8's lecture).
- `verify_checksum` returns a True or False value, based on if the segments stored checksum matches the recalculated checksum of the payload (to ensure no corruption has occurred).
- `create_segment` constructs and returns a new Segment object with provided header fields and payload.

On the sending side of the layer, the function splits the outgoing data into 500 (or less) byte chunks via the `segment_data` function so that each payload is (or is under) the maximum defined size. Next, a UDP checksum is calculated (which will be then verified when received). The function implements an alternating bit system so that the sender retransmits a segment in a loop until an ACK with the correct sequence number arrives.

The receive function for the layer verifies the checksum of an incoming segment, and returns the in-order Data back to the application layer. Finally, the function sends an ACK for the received sequence number to the source IP address.

## Network Layer

The Network layer consists of a send and a receive function, detailing how the respective function is to work. The send function performs a routing table look up using the destination's /24 prefix (used as the routing table key) to determine the IP address of the next hop, as well as the outgoing interface. Next, the segment is encapsulated into a packet with the destination and source IP addresses, as well as the time-to-live value (which is 100 per the project description) and protocol number (17 as per the project description). Finally, this is passed to the Link Layer.

The receive function first checks the destination IP address to see if the packet has reached its final destination or not, determining if it can be delivered locally or sent on the next hop. If the packet has made it to its final destination it is delivered up to the Transport layer. If the IP addresses don't match, it decreases the time-to-live by 1 (checking if it has hit 0 - suggesting the packet needs to be dropped), and performs a routing table look up to find where the next hop the packet is to be forwarded to. Finally, (in this case where the packet is not at its final destination) the packet is forwarded to the Link Layer.

## Link Layer

The Link layer consists of a send and a receive function, detailing how the respective function is to work. The send function first looks up the next hops IP address in the ARP table in order to get the destination's MAC address, and selects the correct source MAC address (routers have per-interface MACs, whereas hosts have a single MAC address). A Frame object is then created to encapsulate the packet with the source and destination MAC addresses, as well as the frame type, packet length and packet (as the frames payload). Finally, this is passed directly to the connected device to simulate a physical link by calling the connected device's link\_receive function.

The receive function accepts an incoming frame, and (by recording the interface for routers, or a boolean value for hosts), learns the source MAC address of the frame. This frame is then decapsulated and the payload is then passed to the devices Network layer.

The three-layer stack (as the application layer did not require explicit coding) defined in the python file cleanly mirrors the required layers to implement the given simulation of a real world networking model, and provides the necessary functionality to be inherited by the two device classes (Host and Router) in the [devices.py](#) python file.

## How a message travels through the stack

A message begins in [main.py](#), where HostA, HostB, and R1 (a router) is initialised from [devices.py](#) and using the defined information in [config.py](#). The connections between the devices are set and hostA.application\_send() is called with the data, ports, and destination IP. This hands the data off to Host A's Transport Layer defined and inherited from [protocol.py](#).

The Transport Layer splits the data into chunks of (at most) 500 bytes, computes a UDP-style checksum for each chunk, and wraps it in a Segment. It then enters a wait loop where it will keep retransmitting the segment until it receives a correctly sequenced ACK

back. The segment is passed down to Host A's Network Layer, which looks up the destination's /24 prefix in the routing table to find the next-hop IP and outgoing interface. The segment is encapsulated into a Packet (with source/destination IPs and a TTL of 100) and passed down to the Link Layer. Host A's Link Layer looks up the next-hop IP in the ARP table to get the destination MAC address, wraps the packet in a Frame, and delivers it directly to Router R1 by calling its `link_receive()` method, simulating the physical link.

The R1's Link Layer receives the frame, records the source MAC, and passes the packet up to its Network Layer. Since the destination IP doesn't match the router's own IP, the Network Layer decrements the TTL, performs another routing table lookup, and forwards the packet back down through its own Link Layer to Host B.

Host B's Link Layer receives the frame and passes the packet up to its Network Layer, which recognises the destination IP as its own and delivers the segment up to the Transport Layer. The Transport Layer verifies the checksum, delivers the data to the application, and sends an ACK Segment back to Host A along the same path in reverse. When the ACK arrives back at Host A's Transport Layer with the correct sequence number, the wait loop stops, the sequence number flips between 0 and 1, and the process repeats for the next chunk until all data has been delivered.